

Más allá del taint: invariantes estructurales y witnesses verificables por máquina para fallas de autorización, ciclos de estado y topologías de confianza

Proyecto VALEN

Contribuciones de investigación — método, resultados medidos y limitaciones

Abstract

Presentamos VALEN — *Verificación y Lógica Activa, Neuro-simbólica* — un sistema para verificar invariantes de seguridad *estructurales* — propiedades de autorización, de transición de estado y de topología de confianza — donde el análisis de taint clásico es demostrablemente ciego porque el dato infractor es legítimo. Hacemos cinco contribuciones medidas. (1) Un *resultado de frontera* riguroso: sobre OWASP Benchmark 1.2 (2740 casos), las invariantes espectrales, topológicas y geométricas no aportan información más allá del taint para priorizar inyecciones (taint MRR 0.894, AUC 0.895; permutación pareada $p = 5 \times 10^{-5}$), lo que zanja la cuestión de si los priors de geometría de grafos ayudan al SAST de inyección lineal. (2) Una invariante de autorización por niveles de privilegio mecanizada en Lean 4, más un detector estructural y un witness de ownership en Z3 para autorización a nivel de objeto (BOLA/IDOR/BFLA) que separa la autorización de objeto de la mera autenticación y recupera todos los endpoints BOLA/BFLA documentados de OWASP crAPI (recall 1.0, precisión 0.90). (3) Evidencia de que la homología de caminos dirigida GLMY es estrictamente mejor que la homología simetrizada para detectar ciclos destructivos (recall 1.0 / precisión 0.75 vs. 0.5 / 0.5). (4) Una frontera de arbitraje neuro-simbólico medida: el LLM cambia la interpretación (28% de CWE), no la detección ni el ranking (acuerdo 0.974). (5) Un análisis de topología de confianza (corte de Fiedler + Forman–Ricci) que aísla puentes de privilegio en grafos de identidades y permisos.

1 Introducción y contribuciones

Los defectos de mayor impacto del software moderno son *estructurales*: un BOLA/IDOR lee el registro de otro usuario con un id entero limpio; una reentrancia es un ciclo destructivo en el grafo de estados; una brecha en la nube es un puente en la topología de confianza; un confused-deputy en un agente es una entrada no confiable que alcanza una herramienta privilegiada sin puerta. En cada caso, el código parece legítimo línea por línea, de modo que el análisis de taint — que rastrea datos “sucios” — es ciego, y los buscadores de patrones no tienen nada que decir. Nuestras contribuciones son:

- C1. Una frontera medida sobre los priors geométricos en SAST de inyección.** Respondemos, con rigor estadístico, si las invariantes espectrales/topológicas/geométricas de los grafos de programa aportan información sobre el taint en fallas de inyección lineal: no aportan. Es un resultado negativo publicable que redirige el esfuerzo lejos de la geometría-sobre-servlets y hacia los dominios donde la estructura es la señal.

- C2. Una invariante de autorización, mecanizada y hecha operacional.** Una función de nivel de privilegio $F : V \rightarrow \mathbb{N}$ con aristas auth $E_A = \{(u, v) : F(u) < F(v)\}$ y el teorema `no_auth_bounded` — un camino sin arista auth no puede escalar privilegio — mecanizado en Lean 4. Operacionalmente: un detector estructural de acceso a recurso controlado por el usuario sin puerta, y un witness en Z3 que demuestra $\text{SAT}(\text{owner}(id) \neq \text{session})$, separando la autorización a nivel de objeto de la mera autenticación.
- C3. La homología de caminos dirigida como la señal de ciclo correcta.** La homología dirigida GLMY recupera el generador concreto β_1 de un ciclo destructivo y, medida sobre 12 grafos de estado, domina a la homología simetrizada — que a la vez pierde 2-ciclos y emite ciclos falsos en triángulos rellenos.
- C4. Una frontera de arbitraje neuro-simbólico medida.** El LLM propone, los métodos formales deciden: la detección y el ranking son independientes del LLM (acuerdo 0.974), mientras el LLM cambia la interpretación (CWE, títulos) en el 28% de los hallazgos.
- C5. Análisis de topología de confianza.** El corte de Fiedler y la curvatura de Forman–Ricci, aplicados a grafos de identidades y permisos, aíslan puentes de privilegio y rupturas de segmentación de confianza para el análisis de movimiento lateral e IAM.

2 Sistema

Una única IR de grafo tipado es el sustrato: seis adaptadores (Python, Java con taint interprocedural/context-sensitive, binario vía `objdump/angr`, OpenAPI, IAM, llm-agent) emiten el mismo grafo determinista y serializable en JSON, compartido literalmente entre las capas de Python y Rust. Cinco señales se fusionan sobre él: algebraica (retículo de taint + niveles de privilegio), espectral (Fiedler), topológica (homología persistente y GLMY dirigida), geométrica (Forman–Ricci / Ollivier–Ricci) y formal (Z3 + Lean 4). Un ciclo neuro-simbólico extrae specs estructuradas y las descarga, emitiendo solo witnesses verificables por máquina.

3 Contribución 1: la frontera sobre los priors geométricos

Configuración. OWASP Benchmark 1.2 provee 2740 casos con verdad de referencia conocida; preguntamos si cada señal ordena el sink vulnerable por encima de los demás. **Método.** Un oráculo de priorización calcula MRR, R@1, NDCG@ k , una cola global sobre *todos* los candidatos (seguros y vulnerables), curvas de costo, un test de permutación pareada e intervalos de confianza bootstrap. **Resultado.** El taint crudo domina: MRR 0.894 frente a 0.872 del campo fusionado ($p = 5 \times 10^{-5}$, add-one sobre 20000 permutaciones, IC 95% [+0.0157, +0.0294]); el AUC espectral (0.496) y topológico (0.500) está en el azar y la curvatura por debajo (0.376). **Interpretación.** Un servlet plano y casi acíclico no tiene curvatura ni topología persistente que explotar, y el taint ya es casi óptimo allí; el resultado es una respuesta falsable y reproducible, no una limitación de la maquinaria.

4 Contribución 2: autorización, mecanizada y operacional

Invariante. Sea $F : V \rightarrow \mathbb{N}$ un nivel de privilegio; una arista $e = (u, v)$ es una *arista auth* si y solo si $F(u) < F(v)$. Entonces (i) un camino sin aristas auth satisface $F(k) \leq F(s)$ — sin escalada de privilegio — y (ii) un flujo de taint que cruza una arista auth sin sanitizador es un cruce de frontera

de privilegio controlado por el atacante. (i) está mecanizado en Lean 4 (`no_auth_bounded`); la solidez de la abstracción booleana de taint está mecanizada como `prop1_soundness`. **Detector.** El detector de autorización-ausente/BOLA marca una función con fuente HTTP y sink de recurso pero sin puerta auth. Sobre un corpus curado de 39 casos alcanza recall 1.0 / precisión 0.72 (F1 0.837) donde el taint puntúa 0; una escalera de baselines muestra que cada requisito rinde ($0.46 \rightarrow 0.72$). **Witness.** El verificador Z3 comprueba $\text{SAT}(\text{owner}(\text{object_id}) \neq \text{session_user_id})$: un endpoint vulnerable produce la sesión del atacante y el objeto de la víctima; un endpoint con `owner` acotado es UNSAT. Hace preciso que `login_required` (autenticación) *no* bloquea BOLA — solo un predicado `owner` lo hace. **Datos reales.** Sobre OWASP crAPI (44 operaciones, 9 endpoints BOLA/BFLA documentados), una señal de referencia a objeto a nivel de API recupera los 9 (recall 1.0, precisión 0.90) mientras el chequeo de auth-ausente puntúa 0, porque todo endpoint BOLA *está* autenticado.

5 Contribución 3: homología de caminos dirigida

Simetrizar un grafo dirigido es pérdida en ambos sentidos: un 2-ciclo $A \rightarrow B \rightarrow A$ colapsa a una sola arista no dirigida (una reentrada *perdida*), y un triángulo relleno se vuelve un ciclo no dirigido falso. La homología de caminos dirigida GLMY resuelve ambos: sobre un corpus etiquetado de 12 grafos alcanza recall 1.0 / precisión 0.75 frente al 0.5 / 0.5 del baseline simetrizado, y devuelve el generador β_1 concreto (las aristas del ciclo) como evidencia. Sus dos falsos positivos son “diamantes” (huecos de path-space: control que converge), no ciclos de realimentación — el H_1 de GLMY es un superconjunto de los ciclos destructivos y, por tanto, una señal de alto recall, necesaria-pero-no-suficiente.

6 Contribución 4: la frontera de arbitraje neuro-simbólico

Sobre un corpus de 38 archivos (29 hallazgos confirmados), la heurística offline y el agente con LiteLLM coinciden en detección y ranking en 37/38 archivos (acuerdo 0.974; la única excepción es una hipótesis no determinista de `compile`), mientras el LLM cambia el CWE en el 28% de los hallazgos y produce títulos más concisos. Una sonda de minería de specs (8 fragmentos) muestra que el LLM mina una precondition plausible en todos los casos (alucinación 0) con 0.5 de recall de tokens. La contribución es una arquitectura — el LLM propone, Z3 decide — más su medición.

7 Contribución 5: análisis de topología de confianza

Un grafo de identidades y permisos (usuarios, roles, políticas de confianza, servicios) es ingerido por el adaptador IAM y analizado con el corte de Fiedler y la curvatura de Forman–Ricci: el corte saca a la superficie los cruces de frontera de confianza, y una arista con κ_F fuertemente negativa aísla el puente de privilegio — el único conector entre dos subredes de confianza. Sobre un grafo de demostración, el `assume app→data` se marca como el puente más negativo ($\kappa_F = -1$) y el bucket de secretos aparece en la frontera de Fiedler; Z3 resuelve entonces la combinatoria de asunción de roles.

8 Asistencia red-team

VALEN no es solo un verificador: un hallazgo más un witness se convierte en un *artefacto ejecutable* para equipos ofensivos. La capa red-team convierte cada candidato a nivel de objeto en un proof-

of-concept “requests” ejecutable (`valen.redteam.poc`) que un pentester corre con un token de atacante y el id de objeto de una víctima, y un *validador activo* (`valen.redteam.live`) que replayea la petición contra un host accesible y confirma el leak en la respuesta — cerrando el ciclo de “satisfacible bajo el modelo” a “impacto demostrado, acá está la evidencia”. Sobre crAPI esto produce un PoC por endpoint BOLA/BFLA documentado (9), basado en el mismo spec y las mismas etiquetas usadas para la verificación.

Más allá de hallazgos individuales, VALEN arma un *plan de ataque ordenado* (`valen.redteam.attack_paths`, `valen.redteam.mitre`). Los hallazgos y las relaciones IAM se etiquetan con tácticas de MITRE ATT&CK; las cadenas de escalada sobre el grafo de identidades/permisos se computan como secuencias ordenadas de aristas trust/assume/access, cada paso confirmado por un encoding SMT de la propagación de “attacker_has(role)”. El resultado es un plan en orden de kill-chain Discovery → Privilege Escalation → Collection, donde cada paso lleva su táctica, el PoC concreto y la confirmación SMT — de modo que un equipo red-team recibe un plan priorizado y respaldado por evidencia, en lugar de una lista de alertas.

Por último, VALEN ingiere la salida de reconocimiento de Kali (`valen.redteam.recon`): `nmap -oX` y `masscan -oJ` se convierten en un grafo de superficie de ataque (hosts y servicios abiertos como nodos, aristas host → servicio), que los mismos kernels de Fiedler/Forman–Ricci convierten en puentes de red y que una tabla de versiones → CVE convierte en hipótesis para el ciclo. El sigilo es una preocupación explícita y de primera clase (`valen.redteam.stealth`): timing conservador (–T0..–T2), límites de tasa, orden de hosts aleatorio, jitter entre sondas, padding y modos solo-pasivos — para engagements autorizados, sin escaneo implícito. Más allá de hallazgos individuales, VALEN realiza *enumeración* (`valen.redteam.enum`: salida de gobuster/ffuf/amass → IR, con señal de *shadow API* para endpoints ausentes del spec OpenAPI) y *explotación activa* (`valen.redteam.auth/jwt/idor`): una forja de JWT con stdlib (“kid” path-traversal, `alg=none`, firma inválida) logra *account takeover* contra un lab local de crAPI, y la enumeración activa de ids confirma BOLA/IDOR con evidencia request/response — todo dry-run por defecto y bloqueado tras un `–authorize –scope` explícito. Los hallazgos, cada uno con su puntaje base CVSS 3.1, se emiten como reporte HTML autocontenido y se renderizan a PDF con Chrome headless (`valen.redteam.report`).

El mismo stack se empaqueta como un *agente autónomo de pentest* (`valen.redteam.operators/executor/cha`) operadores por tiers (passive/bounded/intrusive), un planner dirigido por objetivos con audit log y fallback LLM, y un harness que re-ejecuta cada uno de los 18 challenges documentados de crAPI contra un lab limpio. Resuelve un subconjunto de forma autónoma (BOLA/IDOR, mass assignment, BFLA, forja de JWT, exposición excesiva, inyección NoSQL, acceso no autenticado, reset de contraseña, SSRF) y reporta el resto honestamente. Finalmente, la tesis estructural se extiende a *smart contracts* (`valen.ingest.solidity` + `valen.analysis.reentrancy`): la reentrancia es una violación checks-effects-interactions que también forma un 2-ciclo dirigido; sobre SmartBugs-curated (31 positivos / 112 negativos) el detector de orden alcanza F1 0.80, la homología dirigida GLMY recuerda 0.97, y la simetrizada colapsa a 0.065 — la misma brecha dirigida-vs-simetrizada mostrada para máquinas de estado, ahora en contratos reales.

9 Limitaciones

La evaluación BOLA/IDOR es una sola API real más un corpus curado, no un benchmark; el detector estructural sobre-aproxima (casos sanitizados/ parametrizados); el witness de Z3 vale bajo la semántica *modelada*, no frente a la explotabilidad end-to-end; y la señal GLMY es necesaria pero no suficiente. Cada una se reporta, no se oculta.

10 Conclusión

VALEN contribuye un resultado de frontera falsable, una invariante de autorización mecanizada y operacional, evidencia de que la homología dirigida es la señal de ciclo correcta, una frontera de arbitraje neuro-simbólico medida y un análisis de topología de confianza — todo sobre una única IR de grafo tipado, y todo emitiendo witnesses verificables por máquina. En conjunto argumentan que el rol productivo de las invariantes estructurales en seguridad es la *verificación* de propiedades de autorización, estado y confianza, no la priorización de sinks de inyección.